

Lab4

Course: ARTI 404 – Image Processing

Lab Topic: Intensity Transformations and Filtering-Spatial Domain

Student Name: Muhannad Almutlaq

Student ID: 2240006060

1. Lab Outcomes

The objective of this lab is to write a program that implements fundamental image processing algorithms, specifically image thresholding and histogram processing techniques.

2. Procedural Tasks

Task 1: Thresholding

- **Description:** Loaded the 'Parrot.png' image in grayscale and converted it into a binary image using fixed threshold values of 0, 50, 100, 150, and 200 to separate foreground from background. Each thresholded image is also saved to the outputs folder with cv2.imwrite.
- **Code & Output:**

```
import cv2
import matplotlib.pyplot as plt

# Load the image in grayscale
image= cv2.imread("./images/Parrot.png", cv2.IMREAD_GRAYSCALE)

# Set the threshold values
threshold_values= [0, 50, 100, 150, 200]

# Create a figure with a 2x3 grid of subplots (for 1 original + 5 thresholded images)
fig, axes= plt.subplots(2, 3, figsize= (15, 10))

# Flatten the axes array to make indexing easier in the loop
axes= axes.flatten()

# Display the original image in the first subplot
axes[0].imshow(image, cmap= "gray")
```

```

axes[0].set_title("Original Image")
axes[0].axis("off")

# Apply thresholding with different threshold values
for i, threshold_value in enumerate(threshold_values):
    ret, thresh = cv2.threshold(image, threshold_value, 255, cv2.THRESH_BINARY)
    cv2.imwrite(f"./outputs/Threshold_{threshold_value}.png", thresh)

    # Display in subplot (i+1 because 0 is the original image)
    axes[i + 1].imshow(thresh, cmap= "gray")
    axes[i + 1].set_title(f"Threshold: {threshold_value}")
    axes[i + 1].axis("off")

plt.tight_layout()
plt.show()

```



Task 2: Histogram Processing (Contrast Stretching)

- **Description:** Loaded the `skimage.data.moon()` image and applied contrast stretching to rescale the intensity values between the 2nd and 98th percentiles. The first figure shows the original image with its histogram, and the second shows the stretched image with its histogram.
- **Code & Output:**

```
import matplotlib
import matplotlib.pyplot as plt
import numpy as np
from skimage import data, img_as_float
from skimage import exposure

image= data.moon()

p2, p98 = np.percentile(image, (2, 98))
image_rescale= exposure.rescale_intensity(image, in_range= (p2, p98))

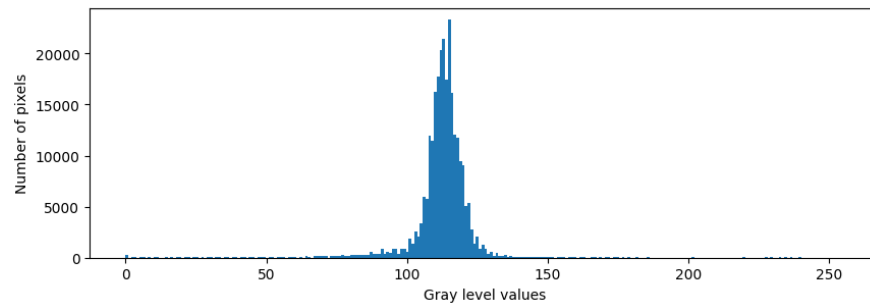
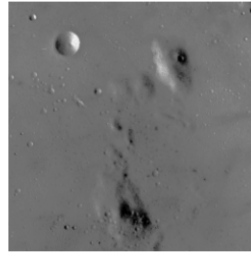
fig= plt.figure(figsize= (10, 7))
fig.add_subplot(2, 1, 1)
plt.imshow(image, cmap= "gray")
plt.axis("off")
plt.title("Original image")

fig.add_subplot(2, 1, 2)
plt.hist(image.flat, bins= 256, range= (0, 255))
plt.xlabel("Gray level values")
plt.ylabel("Number of pixels")
plt.show()

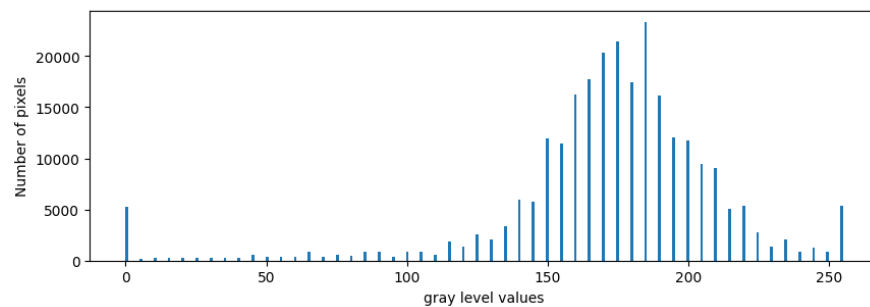
fig= plt.figure(figsize= (10, 7))
fig.add_subplot(2, 1, 1)
plt.imshow(image_rescale, cmap= "gray")
plt.axis("off")
plt.title("Contrast stretch")

fig.add_subplot(2, 1, 2)
plt.hist(image_rescale.flat, bins= 256, range= (0, 255))
plt.xlabel("gray level values")
plt.ylabel("Number of pixels")
plt.show()
```

Original image



Contrast stretch



3. Lab Assessment

Assessment Task 1: Contrast Stretching (3rd and 80th percentiles)

- **Objective:** Rescale the intensity values of the moon image to include the intensities falling within the 3rd and 80th percentiles, and plot the histogram.
- **Code & Output:**

```
import numpy as np
```

```
import matplotlib.pyplot as plt
from skimage import data, exposure

# Load the moon image
img_moon= data.moon()

# Calculate the 3rd and 80th percentiles
p3, p80 = np.percentile(img_moon, (3, 80))

# Rescale intensity values
img_rescale= exposure.rescale_intensity(img_moon, in_range= (p3, p80))

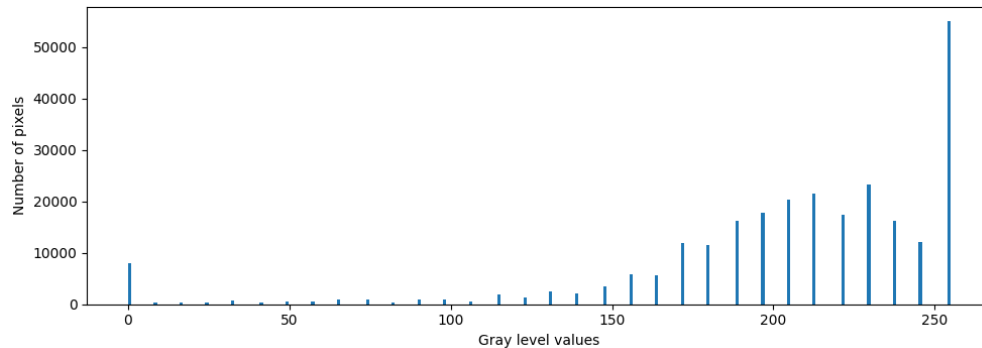
# Plot the rescaled image and its histogram
fig= plt.figure(figsize= (10, 7))

# Image subplot
fig.add_subplot(2, 1, 1)
plt.imshow(img_rescale, cmap= "gray")
plt.axis("off")
plt.title("Contrast stretch (3rd to 80th percentile)")

# Histogram subplot
fig.add_subplot(2, 1, 2)
plt.hist(img_rescale.flat, bins= 256, range= (0, 255))
plt.xlabel("Gray level values")
plt.ylabel("Number of pixels")

plt.tight_layout()
plt.show()
```

Contrast stretch (3rd to 80th percentile)



Assessment Task 2: Histogram Equalization

- **Objective:** Use the moon image and the `exposure.equalize_hist` function to display the image and the histogram after flattening. `equalize_hist` outputs an image with float values from 0.0 to 1.0, so the histogram range is set to (0, 1).
- **Code & Output:**

```
image_eq= exposure.equalize_hist(img_moon)

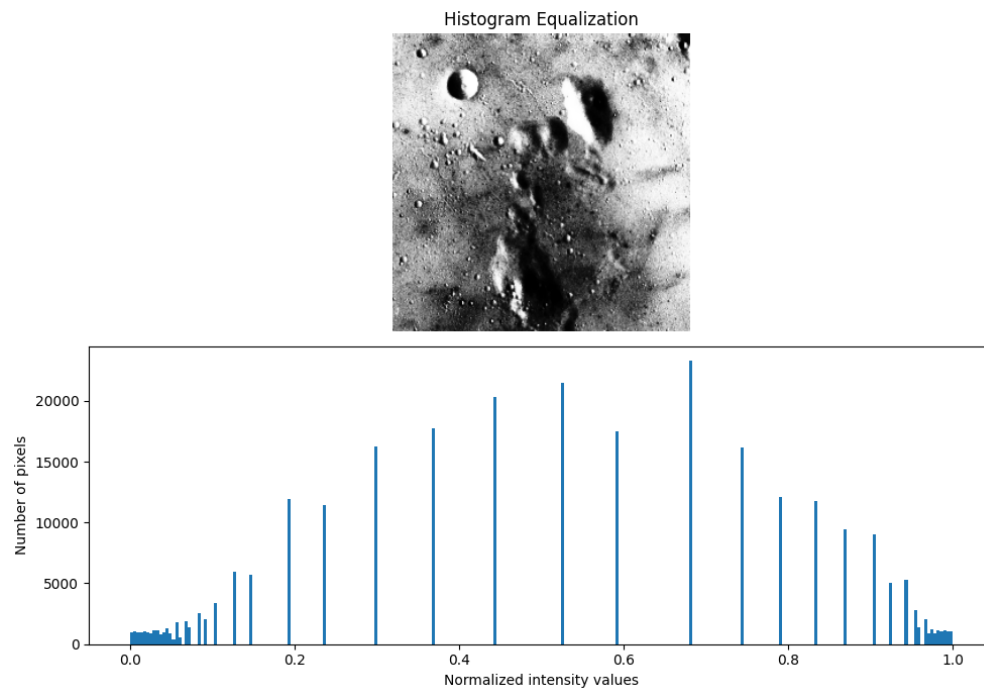
# Plot the equalized image and its histogram
fig= plt.figure(figsize= (10, 7))

# Image subplot
fig.add_subplot(2, 1, 1)
plt.imshow(image_eq, cmap= "gray")
plt.axis("off")
plt.title("Histogram Equalization")

# Histogram subplot
fig.add_subplot(2, 1, 2)
plt.hist(image_eq.flat, bins= 256, range= (0, 1)) # The range is set to (0, 1) since
the intensities are normalized floats
plt.xlabel("Normalized intensity values")
plt.ylabel("Number of pixels")

plt.tight_layout()
```

```
plt.show()
```



Assessment Task 3: Histogram Matching

- **Objective:** Implement histogram matching using the Chelsea image as the source and the rocket image as the reference. Both are color (RGB) images, so `channel_axis= -1` makes the matching work on each color channel separately.
- **Code & Output:**

```
from skimage.exposure import match_histograms

# Load the images
image_chelsea= data.chelsea() # Source image
image_rocket= data.rocket() # Reference image

# Implement histogram matching
matched= match_histograms(image_chelsea, image_rocket, channel_axis= -1)

# Plotting the results
fig, axes = plt.subplots(nrows= 1, ncols= 3, figsize= (15, 5))

axes[0].imshow(image_chelsea)
axes[0].set_title("Source Image (Chelsea)")
axes[0].axis("off")
```

```
axes[1].imshow(image_rocket)
axes[1].set_title("Reference Image (Rocket)")
axes[1].axis("off")

axes[2].imshow(matched)
axes[2].set_title("Matched Image")
axes[2].axis("off")

plt.tight_layout()
plt.show()
```

Source Image (Chelsea)



Reference Image (Rocket)



Matched Image

